

AI Dev Assistant

Architecture & Flow

Extension internals, cache design, and end-to-end analysis pipeline

1. Overview

AI Dev Assistant is a VS Code extension that continuously watches JavaScript, TypeScript, and Java files for edge-case and business-logic bugs. It runs entirely inside the VS Code extension host — no backend server, no external process. Analysis is split into two tiers to give instant local feedback while reserving LLM calls only for functions that static analysis cannot reason about.

Design Principles

- Tier-1 always renders before Tier-2 is invoked — responsiveness never depends on network latency.
- LLM output is always structured JSON — results map directly to `vscode.Diagnostic` without free-text parsing.
- 5-second timeout on LLM calls — falls back to Tier-1 only, never hangs the editor.
- Tier-2 is skipped entirely if Tier-1 already flagged the function — saves API quota.
- Tier-1 works fully offline — no internet required for static analysis.
- Output is suggest-only — no auto-fix, no auto-apply.

2. Cache Storage

The extension maintains three in-memory caches, all living in the VS Code extension host Node.js process. None are persisted to disk. All reset on VS Code restart or via the "AI Dev Assistant: Clear Analysis Cache" command.

Cache 1 — LLM Response Cache (`src/tier2/hashCache.ts`)

```
const cache = new Map<string, LLMResponse>();
//           ^           ^
//   key: SHA-256 of function text   value: parsed LLM JSON response
```

- Key: SHA-256 hex hash of the function's full source text (Node crypto module).
- Value: `LLMResponse` — `{ issues: LLMIssue[] }` with line, severity, message, suggestedFix.
- A cache hit means the LLM is never called again for an unchanged function across saves.
- The hash changes only if the function body changes; any edit triggers a fresh LLM call.
- Size: unbounded Map — grows until reloaded or cleared via the command.

Cache 2 — Diagnostic Lifecycle Cache (`extension.ts`)

```
const llmDiagnosticsByHash = new Map<
  string,                // file path (absolute)
  Map<string,            // function SHA-256 hash
    vscode.Diagnostic[]> // LLM diagnostics for that function
>();
```

- Outer key: absolute file path. Inner key: function hash.
- On every save, functions no longer present are pruned — prevents ghost diagnostics.
- `publishDiagnostics()` merges `tier1Diagnostics` + all `llmStore` values atomically.

Cache 3 — Function Map (`extension.ts`)

```
const prevFunctionMap = new Map<string, ParsedFunction[]>();
//           ^file path ^last-seen functions
```

- Enables `diffFunctions()` — only changed functions are forwarded to Tier-2.
- On first open, `prev` is empty so all functions run through the full pipeline.

3. Repository Structure

<code>src/extension.ts</code>	Activation entry, event wiring, diagnostic lifecycle
<code>src/tier1/astParser.ts</code>	ts-morph in-memory project; extracts <code>ParsedFunction[]</code> from JS/TS
<code>src/tier1/javaAdapter.ts</code>	java-parser CST walker; extracts Java methods (pure JS, no JDK)
<code>src/tier1/ruleEngine.ts</code>	7 deterministic JS/TS rules
<code>src/tier1/javaRuleEngine.ts</code>	7 deterministic Java rules
<code>src/tier1/languageAdapter.ts</code>	Dispatcher: <code>java !' javaAdapter/javaRuleEngine</code> , else <code>!' astParser/ruleEngine</code>
<code>src/tier1/diagnosticMapper.ts</code>	Maps <code>RuleViolation[]</code> + <code>LLMIssue[]</code> <code>!' vscode.Diagnostic[]</code> with line offsets
<code>src/tier2/interestFilter.ts</code>	Business-logic keyword filter — decides if LLM is warranted
<code>src/tier2/contextBuilder.ts</code>	Extracts <code>FunctionContext</code> (name, code, params, return type, docstring)
<code>src/tier2/promptTemplate.ts</code>	JSON-schema-constrained prompt builder
<code>src/tier2/hashCache.ts</code>	SHA-256 keyed in-memory Map for LLM responses
<code>src/tier2/orchestrator.ts</code>	<code>callWithFallback()</code> — Groq <code>!' OpenRouter <code>!' Gemini</code> with timeout</code>
<code>src/providers/groq.ts</code>	Primary: llama-3.1-8b-instant, JSON mode, 5s timeout
<code>src/providers/openRouter.ts</code>	Fallback 1 — OpenRouter API
<code>src/providers/gemini.ts</code>	Fallback 2 — Google Gemini free tier
<code>src/providers/rateLimiter.ts</code>	Client-side token bucket per provider
<code>src/chat/chatParticipant.ts</code>	@devassistant VS Code Chat handler, streamed markdown
<code>scripts/pipeline-dev.ts</code>	Standalone harness — tests 4 scenarios outside VS Code
<code>test/scenario-a.ts</code>	Fixture: <code>getEmail</code> — null access (Tier-1)
<code>test/scenario-b.ts</code>	Fixture: <code>applyDiscount</code> — discount > 100% (Tier-2)
<code>test/scenario-c.ts</code>	Fixture: <code>updateOrderStatus</code> — illegal transition (Tier-2)
<code>test/scenario-d.java</code>	Fixture: <code>hasRole</code> — <code>String ==</code> comparison (Tier-1 Java)

4. End-to-End Flow on File Open / Save

Step 1 — Trigger

onDidOpenTextDocument fires when a file is opened. onDidSaveTextDocument fires on Ctrl+S. isSupportedLanguage() checks the languageId: javascript, typescript, javascriptreact, typescriptreact, java.

Step 2 — AST Parsing

- JS/TS: ts-morph creates an in-memory Project, extracts FunctionDeclaration, MethodDeclaration, ArrowFunction, FunctionExpression.
- Java: java-parser (ANTLR-based, pure JS) produces a CST; javaAdapter walks it recursively to find method nodes.
- Each function gets a SHA-256 hash of its text — the stable identity key for both caches.

Step 3 — Change Detection

```
diffFunctions(prev, current)  !'  only functions whose hash changed since last save

// Keyed on (name + startLine) to handle overloaded and anonymous functions
prevMap.get(`${f.name}:${f.startLine}`) !== f.hash  !'  "changed"
```

Step 4 — Tier-1 (synchronous, instant)

runRulesByLanguage() is called for ALL current functions. Results are mapped to vscode.Diagnostic[] and published immediately via diagnosticCollection.set() — squiggles appear before any LLM call.

Step 5 — Tier-2 Gate (four conditions)

```
1.  changed.length > 0           // at least one function changed
2.  enableTier2 setting is true   // not disabled in VS Code settings
3.  isInteresting(fn) === true    // name/body contains a business-logic keyword
4.  tier1HitsByHash.get(fn.hash)  // Tier-1 found ZERO violations for this function
    === 0                        // !' if Tier-1 flagged it, LLM is skipped entirely
```

Step 6 — Interest Filter

Checks function name and body for business-logic keywords:

```
calculate, calc, validate, apply, transition, process, compute,
charge, price, discount, cost, fee, order, status,
auth, authorize, permission, role, limit, quota, threshold,
balance, payment, refund, schedule, expire
```

Step 7 — Hash Cache Lookup

hashCache.get(fn.hash) — a hit reuses the cached LLMResponse immediately. A miss proceeds to callWithFallback().

Step 8 — LLM Call + Fallback

```
Groq (llama-3.1-8b-instant) % % %° primary (json_mode enforced)
! " fail / timeout
OpenRouter % % % % % % % % % % % % %° fallback 1
! " fail / timeout
Gemini free tier % % % % % % % % % % %° fallback 2
! " all failed
throw Error % % % % % % % % % % % % %° showWarningMessage("Open Settings")
```

Step 9 — Response Parsing & Diagnostic Publishing

```
LLM returns: { "issues": [{ "line": 3, "severity": "error",  
                             "category": "value-invariant", "message": "...", "suggestedFix": "..."}] }  
  
// issue.line is 1-based and relative to function start  
absoluteLine(0-based) = fnStartLine + issue.line - 2  
  
// Merged and published atomically:  
diagnosticCollection.set(uri, [...tier1Diagnostics, ...llmStore.values().flat()])
```

5. Tier-1 Rules

JavaScript / TypeScript (src/tier1/ruleEngine.ts)

null-access

Detects property chains depth "e 3 without optional chaining where the intermediate property is not null-guarded.

```
TRIGGERS: user.profile.email    (no guard)
SAFE:     user?.profile?.email  (optional chaining)
```

array-bounds

Detects element access with a non-literal index outside any conditional guard.

```
TRIGGERS: arr[i]    (no length check)
SAFE:     i < arr.length ? arr[i] : undefined
```

division-by-zero

Detects division (/) or modulo (%) where the denominator is a variable, not a non-zero literal.

```
TRIGGERS: total / count  (count may be 0)
SAFE:     total / 2      (literal non-zero)
```

unhandled-promise

Detects await expressions not enclosed in a try/catch within their immediately enclosing async function.

```
TRIGGERS: async fn() { const r = await fetch(url); }
SAFE:     async fn() { try { await fetch(url); } catch(e) {} }
```

loose-equality

Detects == and != operators that allow implicit type coercion.

```
TRIGGERS: x == y,  x != null
FIX:      x === y, x !== null
```

off-by-one

Detects for loops using <= with .length — the last iteration accesses an out-of-bounds index.

```
TRIGGERS: for (let i = 0; i <= arr.length; i++)
FIX:      for (let i = 0; i < arr.length; i++)
```

unvalidated-params

Detects functions with "e 2 parameters that contain no if statement or throw — no input validation present.

```
TRIGGERS: function process(a, b) { return a + b; }
SAFE:     function process(a, b) { if (!a) throw ...; return a + b; }
```

Java (src/tier1/javaRuleEngine.ts)

- java-null-deref — chained method call (a.getB().getC()) without null guard
- java-array-bounds — arr[i] access without a length check
- java-division-by-zero — division/modulo with variable denominator
- java-unchecked-exception — RuntimeException subclass thrown without documentation
- java-string-equals — String comparison with == instead of .equals()
- java-off-by-one — for loop with <= array.length condition
- java-unvalidated-params — method with "e 2 params and no input validation

6. Configuration

<code>enableTier2</code>	boolean, default true — toggle LLM analysis on/off
<code>llmTimeout</code>	number, default 5000 — per-provider timeout in ms
<code>groqApiKey</code>	Groq API key — primary provider. Falls back to GROQ_API_KEY env var
<code>openRouterApiKey</code>	OpenRouter key — fallback 1. Falls back to OPENROUTER_API_KEY env var
<code>geminiApiKey</code>	Gemini key — fallback 2. Falls back to GEMINI_API_KEY env var

API Key Resolution Order (at extension activation)

1.

`process.env.GROQ_API_KEY`

(env var — must be set before VS Code starts)
2.

`aiDevAssistant.groqApiKey`

(VS Code settings: Ctrl+, !' search groqApiKey)

Tip: use VS Code settings if the env var is not visible to the VS Code process.

7. Demo Scenario Matrix

All four scenarios can be verified with: `pnpm run harness`

Scenario	Function	Tier	Rule	Bug detected
A	<code>getUserEmail(user)</code>	Tier-1	null-access	<code>user.profile.email</code> — profile may be undefined
B	<code>applyDiscount(price, pct)</code>	Tier-2	value-invariant	<code>discountPercent > 100</code> !' negative price
C	<code>updateOrderStatus(customer, new)</code>	Tier-2	state-invariant	refunded !' shipped is illegal transition
D (Java)	<code>hasRole(role, req)</code>	Tier-1	java-string-equals	<code>String ==</code> compares references, not values